

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics
COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I JAGADISH (192521352)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: JAGADISH.V

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Experiment 1: Cloud Storage Comparison

Aim:

To compare Object, Block, and File Storage and select the appropriate storage model for backups, VM disks, and shared enterprise documents.

Storage Type	Description	Best Use
Object Storage	Stores data as objects with metadata	Backups, images, videos, archives
Block Storage	Provides raw storage blocks attached to virtual machines	VM disks, databases
File Storage	Provides shared files through a file system	Enterprise documents, shared folders

Example Cloud Services:

- Object: Amazon S3 / Azure Blob Storage
- Block: Amazon EBS / Azure Managed Disks
- File: Amazon EFS / Azure Files

Selection:

- **Backups** → **Object Storage**
- **VM Disks** → **Block Storage**
- **Shared Enterprise Documents** → **File Storage**

Result:

Object, block, and file storage were compared successfully, and the appropriate storage model was identified for each requirement.

Experiment 2: REST API Workflow for Cloud Student Portal

Aim:

To develop and demonstrate a simple REST API workflow for a cloud-based student portal.

Workflow

Student



Web Browser / Mobile App



HTTPS Request



Authentication



Cloud REST API



Database



JSON Response



Student Portal

Example Request

GET /api/students/101

Authorization: Bearer <token>

Example Response

```
{
  "id": 101,
  "name": "Student",
  "course": "Information Technology",
```

```
"cgpa": 8.14
}
```

Procedure

1. Create a cloud-based student portal.
2. Define REST API endpoints such as:
 - GET /students
 - GET /students/{id}
 - POST /students
 - PUT /students/{id}
3. Implement authentication using a token.
4. Send HTTP requests from the client.
5. Validate the authentication token.
6. Access student information from the cloud database.
7. Return the result in JSON format.
8. Deploy the API to a cloud platform.

Result:

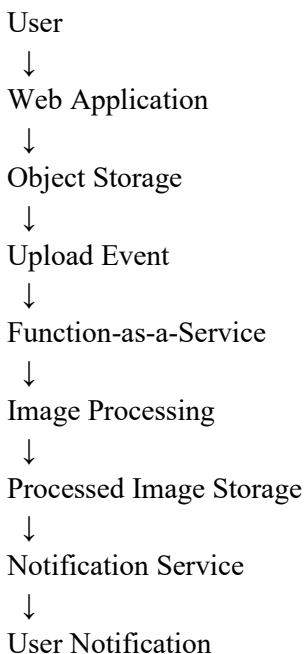
A REST API workflow containing request, authentication, cloud processing, database access, and JSON response was successfully demonstrated.

Experiment 3: Serverless Image Upload Processing

Aim:

To design a serverless event-driven workflow for automatically processing uploaded images.

Architecture



Procedure

1. Create an object-storage bucket.
2. Allow users to upload images.
3. Configure an upload event.
4. Connect the event to a serverless function.
5. The function is automatically triggered.
6. Perform image processing such as resizing or format conversion.
7. Store the processed image.
8. Send a notification to the user.

Example Services:

Function	AWS Example
Object Storage	Amazon S3
Serverless Function	AWS Lambda
Notification	Amazon SNS
Monitoring	Amazon CloudWatch

Result:

A serverless event-driven image processing architecture was designed successfully without requiring a continuously running server.

Experiment 4: Edge Computing vs Fog Computing

Aim:

To compare Edge and Fog Computing for a smart traffic management system based on architecture, latency, and deployment location.

Edge Computing

Traffic Sensors



Edge Device



Traffic Signal

Processing occurs **very close to the data source**.

Fog Computing

Traffic Sensors



Local Fog Gateway



Fog Node



Cloud

Processing is distributed across **local gateways and intermediate network nodes**.

Comparison

Feature	Edge Computing	Fog Computing
Processing Location	Directly at/near device	Between devices and cloud
Latency	Very low	Low
Bandwidth Usage	Very low	Low
Deployment	Traffic camera/signal controller	Local gateway/roadside server
Best Application	Immediate traffic signal control	City-level traffic coordination

Smart Traffic Example

For an emergency vehicle, an **edge device** can immediately detect the vehicle and change the nearby traffic signal. Fog nodes can combine information from several intersections to optimize traffic flow across a larger area.

Result:

Edge computing was found suitable for immediate local traffic decisions, while fog computing is suitable for coordinating multiple traffic systems with low latency.

Experiment 5: Cloud Application Deployment Workflow

Aim:

To demonstrate a complete cloud application deployment workflow involving client access, browser interaction, web API, storage, and monitoring.

Architecture

Client



Web Browser



Internet



Cloud Web Server



Web API



Cloud Storage / Database



Response



Web Browser

Monitoring

Cloud Application



Monitoring Service



CPU / Memory / Requests / Errors



Alerts and Logs

Procedure

1. Develop a simple web application.
2. Create the frontend using HTML, CSS, and JavaScript.
3. Create a web API for application requests.
4. Configure cloud storage/database.
5. Deploy the application to a cloud server or platform.
6. Open the application using a browser.
7. Browser sends a request to the web API.
8. API accesses the required cloud storage/database.
9. The response is returned to the browser.
10. Enable cloud monitoring and logging.
11. Monitor requests, errors, CPU, and memory usage.

Result:

The complete cloud application deployment workflow from client access to API, storage access, response, and monitoring was successfully demonstrated.